

API Automation Playwright

A comprehensive API automation testing framework built with **Playwright** and **TypeScript**. This project provides a robust, maintainable solution for automating API testing with built-in utilities for authentication, request handling, logging, and test data generation.

Table of Contents

- Project Overview
 - Features
 - Prerequisites
 - Project Structure
 - Installation
 - Configuration
 - Running Tests
 - Test Cases Documentation
 - Architecture & Components
 - Dependencies
 - Contributing
 - License
-

Project Overview

API Automation Playwright is a test automation framework designed for testing RESTful APIs using Playwright's powerful API testing capabilities. It leverages TypeScript for type safety and includes utilities for:

- **Authentication Management:** Token-based authentication with caching
 - **API Request Handling:** Wrapper classes for GET, POST, PUT, and DELETE operations
 - **Test Data Generation:** Dynamic test data creation using Faker.js
 - **Structured Logging:** Comprehensive logging system for test execution tracking
 - **Configuration Management:** Environment-based configuration using dotenv
-

□ Features

- **Type-Safe Testing:** Full TypeScript support with strict typing
 - **Reusable API Utilities:** Modular classes for API operations
 - **Token Caching:** Efficient authentication token management
 - **Fake Data Generation:** Realistic test data generation using Faker.js
 - **Detailed Logging:** Comprehensive logging of all API interactions
 - **Environment Configuration:** Easy configuration management via `.env` file
 - **HTML Reporting:** Built-in Playwright HTML test reports
 - **Parallel Test Support:** (Configurable) Ability to run tests in parallel
 - **CI/CD Ready:** Configuration for continuous integration pipelines
-

📦 Prerequisites

Before setting up the project, ensure you have the following installed:

- **Node.js** (v14 or higher) - Download Node.js
 - **npm** (v6 or higher) - Usually comes with Node.js
 - **TypeScript** (v4.0 or higher)
 - A REST API server running (default: `http://localhost:3000`)
-

□ Project Structure

```
API_Automation_Playwright/
├── src/                                # Source code directory
│   ├── apiUtils/                      # API utility classes
│   │   ├── auth.ts                   # Authentication utilities
│   │   └── requests.ts               # HTTP request wrappers (GET, POST, PUT, DELET
E)
│   ├── dataGenerator/                # Test data generation
│   │   └── dataGenerator.ts          # Faker-based user data generator
│   ├── endPoints/                    # API endpoint definitions
│   │   └── endpoints.ts              # Centralized endpoint configuration
│   ├── logUtils/                     # Logging utilities
│   │   └── log.ts                    # Custom logging class
│   └── schema/                       # API schema/model definitions (future use)
├── tests/                             # Test files directory
│   └── APITest.spec.ts               # Main test specification file
├── playwright-report/                 # Generated HTML test reports
│   └── index.html                     # Test report index
├── test-results/                      # Test execution results
├── .env                              # Environment variables
├── .gitignore                         # Git ignore rules
├── .github/                           # GitHub-related configurations
├── package.json                       # Project dependencies and metadata
├── package-lock.json                  # Locked dependency versions
├── playwright.config.ts                # Playwright configuration
└── README.md                          # This file
```

□ Installation

Step 1: Clone or Navigate to Project Directory

```
cd API_Automation_Playwright
```

Step 2: Install Dependencies

Install all required npm packages:

```
npm install
```

This will install:

- `@playwright/test` - Playwright testing framework
- `@faker-js/faker` - Fake data generation library
- `dotenv` - Environment variable management
- `@types/node` - TypeScript definitions for Node.js

Step 3: Verify Installation

```
npx playwright --version
npm list
```

⚙️ Configuration

Environment Variables (.env)

Create or update the `.env` file in the project root with the following variables:

```
# Base URL of the API server
baseUrl=http://localhost:3000
```

Available Environment Variables:

Variable	Description	Default Value	Example
<code>baseUrl</code>	Base URL of the REST API server	N/A	<code>http://localhost:3000</code>
<code>CI</code>	Continuous Integration flag (set by CI/CD systems)	<code>undefined</code>	<code>true</code>

Playwright Configuration (playwright.config.ts)

Key configuration options:

```
{
  testDir: './tests',           // Directory containing test files
  fullyParallel: false,         // Disable parallel test execution
  forbidOnly: !!process.env.CI, // Fail if test.only() is used in CI
  retries: process.env.CI ? 2 : 0, // Retries on CI (2 retries), 0 on local
  workers: process.env.CI ? 1 : undefined, // Single worker on CI
  reporter: 'html',             // Generate HTML reports
  trace: 'on-first-retry'       // Capture trace on first retry
}
```

□ Running Tests

Run All Tests

```
npm test
```

or using Playwright directly:

```
npx playwright test
```

Run Specific Test File

```
npx playwright test tests/APITest.spec.ts
```

Run Tests Matching a Pattern

```
npx playwright test -g "Get API"
```

Run with Specific Browser

```
npx playwright test --project=chromium
```

Run Tests in Debug Mode

```
npx playwright test --debug
```

Run Tests in UI Mode (Recommended for Development)

```
npx playwright test --ui
```

View Test Report

After tests complete, view the HTML report:

```
npx playwright show-report
```

Test Cases Documentation

Test Suite: API Tests

Test 1: Get All Users API

Description: Retrieves all users from the API

Method: GET

Endpoint: `/users`

Authentication: Bearer Token (required)

Expected Response: 200 OK with list of users

```
test(`"Get API" : to get all users`, async () => {
  Logger.info(`Running test: ${test.info().title}`);
  await makeRequest.getRequest(USER_ENDPOINTS.getUsers, authToken)
})
```

Test 2: Create New User API

Description: Creates a new user with dynamically generated data

Method: POST

Endpoint: `/users`

Authentication: Bearer Token (required)

Request Body: User object with name, age, and city

Expected Response: 201 Created with new user data

```
test(`"Post API" : to create a new user`, async () => {
  Logger.info(`Running test: ${test.info().title}`);
  await makeRequest.postRequest(USER_ENDPOINTS.createUser, userData, authToken)
})
```

Test 3: Update User API

Description: Updates an existing user (ID: 1)

Method: PUT

Endpoint: `/users/{id}`

Authentication: Bearer Token (required)

Request Body: Updated user object

Expected Response: 200 OK with updated user data

```
test(`"Put API" : to update a user`, async () => {
  Logger.info(`Running test: ${test.info().title}`);
  await makeRequest.putRequest(USER_ENDPOINTS.updateUser(1), userData, authToken)
})
```

Test 4: Delete User API

Description: Deletes a user (ID: 12)

Method: DELETE

Endpoint: `/users/{id}`

Authentication: Bearer Token (required)

Expected Response: 204 No Content or 200 OK

```
test(`"Delete API" : to delete a user`, async () => {
  Logger.info(`Running test: ${test.info().title}`);
  await makeRequest.deleteRequest(USER_ENDPOINTS.deleteUser(12), authToken)
})
```

Architecture & Components

Component Overview

1. Authentication Utilities (`src/apiUtils/auth.ts`)

Handles API authentication with token caching mechanism.

```
class AuthUtils {  
    async authToken(): Promise<string|undefined>  
}
```

Key Features:

- Caches authentication token to avoid multiple login calls
- POST request to `/auth/token` endpoint with credentials
- Returns Bearer token for subsequent API calls
- Logs token retrieval status

Credentials:

- Username: `admin`
 - Password: `admin123`
-

2. API Request Utilities (`src/apiUtils/requests.ts`)

Provides wrapper methods for HTTP operations with built-in logging.

```
class APIUtils {  
    async getRequest(url: string, token?: string): Promise<APIResponse>  
    async postRequest(url: string, data: any, token?: string): Promise<APIResponse>  
    async putRequest(url: string, data: any, token?: string): Promise<APIResponse>  
    async deleteRequest(url: string, token?: string): Promise<APIResponse>  
}
```

Features:

- Automatic Bearer token injection in Authorization header
 - Request/response logging
 - JSON serialization for request bodies
 - Response status and body logging
-

3. Endpoint Definitions (`src/endPoints/endpoints.ts`)

Centralized configuration of all API endpoints.


```
export const USER_ENDPOINTS = {
  auth: '/auth/token',
  getUsers: '/users',
  createUser: '/users',
  updateUser: (id: number) => `/users/${id}`,
  deleteUser: (id: number) => `/users/${id}`,
}
```

Benefits:

- Single source of truth for all endpoints
 - Reusable across multiple test files
 - Dynamic endpoint generation with parameters
-

4. Data Generator (`src/dataGenerator/dataGenerator.ts`)

Generates realistic test data using Faker.js library.

```
interface User {
  name: string;
  age: number;
  city: string;
}

function generateUser(): User
```

Generated Data:

- `name`: Random full name
 - `age`: Random age between 18-60
 - `city`: Random city name
-

5. Logging Utility (`src/logUtils/log.ts`)

Custom logging utility with timestamp and log level support.

```
class Logger {  
    static info(message: string): void  
    static warn(message: string): void  
    static error(message: string): void  
    static debug(message: string): void  
}
```

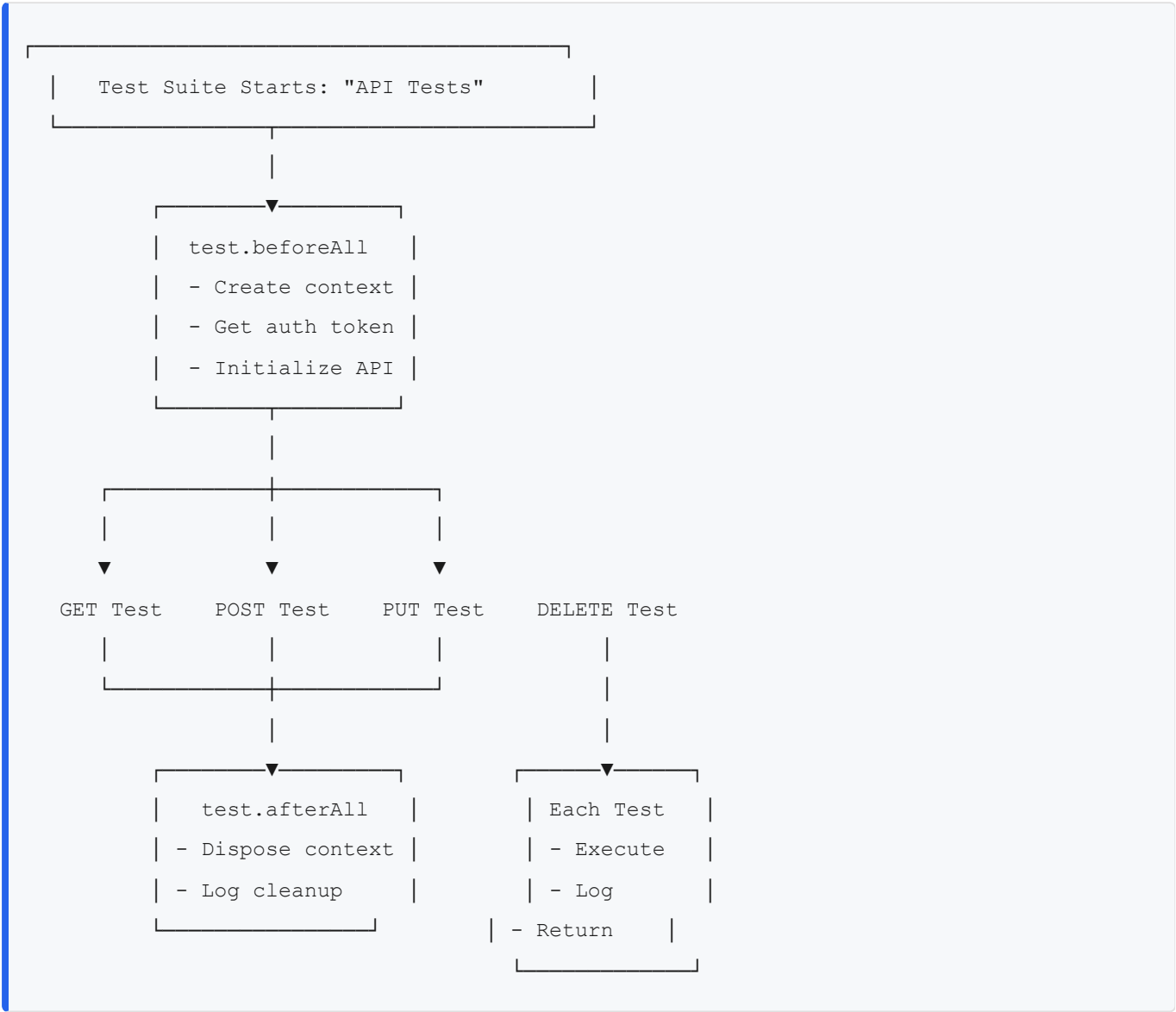
Log Format:

```
[LEVEL] [API_LOGGER] [ISO_TIMESTAMP] [MESSAGE]
```

Example Output:

```
[INFO] [API_LOGGER] 2024-01-15T10:30:45.123Z Token retrieved and stored.
```

Test Execution Flow



Dependencies

Development Dependencies

Package	Version	Purpose
@playwright/test	^1.63.0	Testing framework and API client
@types/node	^26.6.4	TypeScript definitions for Node.js

Runtime Dependencies

Package	Version	Purpose
@faker-js/faker	^10.6.0	Realistic test data generation
dotenv	^18.0.6	Environment variable management

⚙️ Common Commands

```
# Install dependencies
npm install

# Run all tests
npm test
npx playwright test

# Run specific test file
npx playwright test tests/APITest.spec.ts

# Run tests matching pattern
npx playwright test -g "Get API"

# Run in debug mode
npx playwright test --debug

# Run in UI mode
npx playwright test --ui

# View test report
npx playwright show-report

# List installed packages
npm list

# Update dependencies
npm update

# Check for vulnerabilities
npm audit
```

□ Adding New Tests

Step 1: Define New Endpoints

Add new endpoints to `src/endPoints/endpoints.ts`:

```
export const USER_ENDPOINTS = {  
  // ... existing endpoints  
  searchUsers: (query: string) => `/users/search?q=${query}`,  
}
```

Step 2: Create Test Cases

Add test cases in `tests/APITest.spec.ts`:

```
test(`"Search API" : to search users`, async () => {  
  Logger.info(`Running test: ${test.info().title}`);  
  const searchResults = await makeRequest.getRequest(  
    USER_ENDPOINTS.searchUsers("John"),  
    authToken  
  );  
})
```

Step 3: Add Data Models (Optional)

Create schema files in `src/schema/` for complex data structures.

□ CI/CD Integration

The project is pre-configured for CI/CD environments. Environment variables like `CI` are automatically detected:

```
{  
  retries: process.env.CI ? 2 : 0,      // More retries in CI  
  workers: process.env.CI ? 1 : undefined, // Single worker in CI  
  forbidOnly: !!process.env.CI         // Fail if test.only() in CI  
}
```

GitHub Actions Example

```
name: API Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
        with:
          node-version: '18'
      - run: npm install
      - run: npm test
      - uses: actions/upload-artifact@v2
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

📄 Contributing

Guidelines

1. **Create a feature branch:** `git checkout -b feature/new-test`
2. **Write tests:** Add new tests following the existing pattern
3. **Update documentation:** Update README.md if adding new features
4. **Commit changes:** `git commit -m "Add new test case"`
5. **Push to repository:** `git push origin feature/new-test`
6. **Create Pull Request:** Submit for review

Code Standards

- Use TypeScript with strict typing
- Follow existing code style and patterns
- Add comprehensive logging using the Logger class
- Comment complex logic
- Test new features before committing

🔧 Troubleshooting

Issue: Tests fail with "Base URL is not set"

Solution: Ensure `.env` file exists and contains `baseUrl` variable

```
echo "baseUrl=http://localhost:3000" > .env
```

Issue: "Token retrieval failed"

Solution: Verify that:

- API server is running on the configured base URL
- Credentials in `auth.ts` are correct
- `/auth/token` endpoint is accessible

Issue: Tests timeout

Solution: Increase Playwright timeout in `playwright.config.ts`:

```
use: {  
  timeout: 30000, // 30 seconds  
}
```

Issue: Module not found errors

Solution: Reinstall dependencies:

```
rm -rf node_modules package-lock.json  
npm install
```

📖 References & Resources

- **Playwright Documentation:** <https://playwright.dev/docs/intro>
 - **Playwright API Testing:** <https://playwright.dev/docs/api-testing>
 - **Faker.js Documentation:** <https://fakerjs.dev/>
 - **TypeScript Handbook:** <https://www.typescriptlang.org/docs/>
-

📄 License

This project is licensed under the ISC License. See LICENSE file for details.

📝 Author Notes

This framework is designed to be:

- **Maintainable:** Modular architecture with clear separation of concerns
 - **Scalable:** Easy to add new tests and endpoints
 - **Reliable:** Built-in logging and retry mechanisms
 - **Developer-Friendly:** Type-safe with comprehensive documentation
-

📄 Support & Questions

For issues or questions:

1. Check the Troubleshooting section
 2. Review Playwright official documentation
 3. Check test execution logs in `test-results/` directory
 4. View HTML report: `npx playwright show-report`
-

Last Updated: October 2024

Version: 1.0.0

Framework: Playwright with TypeScript